Dr. Alla Jammine
alla.jammine@u-paris.fr

TP 3

CSS Styling & Responsive Design

Objective

The goal of this practical session is to transform the existing HTML page (created in TP2) into a **visually appealing, professional, and fully responsive website** using modern CSS techniques.

Learning Objectives

By the end of this practical, students should be able to:

- Apply **Flexbox and CSS Grid** for layout design
- Implement a **responsive navigation menu**
- Use **media queries** to adapt layouts for mobile and desktop devices
- Integrate **Google Fonts** for professional typography
- *(Optional)* Enhance styling using a CSS framework such as **Bootstrap** or **Tailwind CSS**

This TP work combines:

- Semantic HTML
- Modern CSS layout systems
- Responsive design techniques
- Professional typography and styling

THEORETICAL INTRODUCTION

This laboratory work is based on modern principles of **web layout, styling, and responsiveness**. The following concepts are essential for successful implementation.

1. Separation of Structure and Style

HTML is responsible for the **structure and semantics** of a web page, while CSS controls its **visual presentation**.

- HTML defines *what* the content is
- CSS defines *how* the content looks

Bad practice:

```
<div style="color: red; margin: 20px;">Content</div>
```

Good practice:

```
.content {
  color: red;
  margin: 20px;
```

```
}
```

2. Semantic HTML and Its Role in Styling

Semantic tags improve **code readability, accessibility, and maintainability**.

Common semantic elements:

- `<header>` – page header
- `<nav>` – navigation menu
- `<main>` – main content
- `<section>` – logical content grouping
- `<article>` – independent content block
- `<footer>` – footer area

CSS can directly target these elements:

```
header {
  background-color: #1e1e1e;
}
```

3. CSS Box Model

Every HTML element is represented as a rectangular box.

Components:

- `content`
- `padding`
- `border`
- `margin`

To simplify layout calculations, the following rule is commonly used:

```
* {
  box-sizing: border-box;
}
```

This ensures that padding and borders are included in the element's width and height.

4. Flexbox Layout

Flexbox is designed for **one-dimensional layouts** (row or column).

Key properties:

- `display: flex`
- `flex-direction`
- `justify-content`

- `align-items`

Example (horizontal navigation menu):

```
nav ul {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Flexbox is ideal for:

- Navigation bars
- Centering elements
- Simple responsive layouts

5. CSS Grid Layout

CSS Grid is used for **two-dimensional layouts** (rows and columns).

Key properties:

- `display: grid`
- `grid-template-columns`
- `gap`

Example (main layout):

```
.main-layout {
  display: grid;
  grid-template-columns: 3fr 1fr;
  gap: 20px;
}
```

Grid is suitable for:

- Page layouts
- Content cards
- Dashboards

6. Responsive Design Principles

Responsive design ensures that a website adapts to different screen sizes and devices.

Core techniques:

- Flexible layouts (Flexbox, Grid)
- Relative units (`%`, `rem`, `vw`)
- Media queries

7. Media Queries

Media queries apply CSS rules based on screen size.

Example:

```
@media (max-width: 768px) {
  nav ul {
    flex-direction: column;
  }
}
```

Common breakpoints:

- Mobile: ≤ 768px
- Desktop: ≥ 769px

8. Typography and Google Fonts

Typography significantly affects readability and professional appearance.

Steps to use Google Fonts:

1. Select a font at Google Fonts
2. Import it using `<link>` or `@import`
3. Apply it in CSS

Example:

```
body {
  font-family: 'Inter', sans-serif;
}
```

Use font hierarchy:

- Headings: larger, bolder
- Body text: readable size and weight

9. Visual Consistency and Design Principles

Professional web design relies on:

- Consistent color palette
- Adequate spacing (margin, padding)
- Clear visual hierarchy
- Subtle hover effects

Example:

```
button:hover {
  background-color: #0056b3;
}
```

10. CSS Frameworks (Optional)

Frameworks such as **Bootstrap** or **Tailwind CSS** provide pre-built components and utility classes.

Advantages:

- Faster development
- Responsive by default

Disadvantages:

- Less customization
- Larger CSS size

They should be used **only if the student understands the underlying CSS principles**.

Task Description

You are provided with an **HTML structure created earlier** (including semantic tags, lists, tables, and forms). Your task is to **style this HTML page using CSS** so that it looks modern, clean, and professional, while remaining fully responsive.

Requirements

1. Typography (Google Fonts)

- Import **at least one Google Font** (e.g., *Roboto, Inter, Open Sans, Montserrat*).
- Apply the font consistently across the website.
- Use different font weights and sizes for headings, navigation, and body text.

2. Layout Design

- Use **Flexbox** for:
  - Navigation bar
  - Horizontal or vertical alignment of elements
- Use **CSS Grid** for:
  - Main page layout (e.g., content + sidebar, cards, or sections)

3. Responsive Navigation Menu

- Create a navigation menu that:
  - Displays horizontally on desktop
  - Adapts for mobile screens (stacked menu or toggle-style menu)
- Navigation must remain usable on all screen sizes.

Dr. Alla Jammine
alla.jammine@u-paris.fr

4. Responsive Design (Media Queries)

- Implement **media queries** for:
  - Mobile devices (≤ 768px)
  - Desktop screens
- Adjust layout, font sizes, spacing, and navigation behavior accordingly.

5. Visual Design & Professional Look

- Use a consistent **color palette**
- Add spacing, padding, and margins to improve readability
- Style buttons, links, and forms for a polished appearance
- Avoid inline styles — all styling must be in CSS

6. Code Quality

- Clean, readable CSS
- Proper indentation and comments where necessary
- Logical separation of layout, typography, and components

Optional (Advanced)

- Use **Bootstrap or Tailwind CSS** for part or all of the styling
- Add subtle transitions or hover effects
- Implement a CSS variables (`:root`) theme

Expected Outcome

By the end of the session, each student should have:

- A **fully styled HTML page**
- A **responsive layout** that works on mobile and desktop
- A website that looks **professional, modern, and well-structured**

Result to show:

- Submit:
  - HTML file
  - CSS file(s)
- The page must open correctly in a browser without errors.